

Онлайн-магазин с использованием микросервисной архитектуры

Microservice Architecture



**Меня хорошо видно
& слышно?**



Защита проекта

Тема: Онлайн-магазин с использованием микросервисной архитектуры



Руслан Габбазов

.NET разработчик .С практическими навыками фронтенд-разработки на Vue.js и JavaScript.

План защиты

Цель и задачи проекта

Какие технологии использовались

Что получилось

Выводы

Вопросы и рекомендации

Цель и задачи проекта

1. Создать микросервис
2. Использовать Современные паттерны
3. Внедрить технологии на работе



Какие технологии использовались

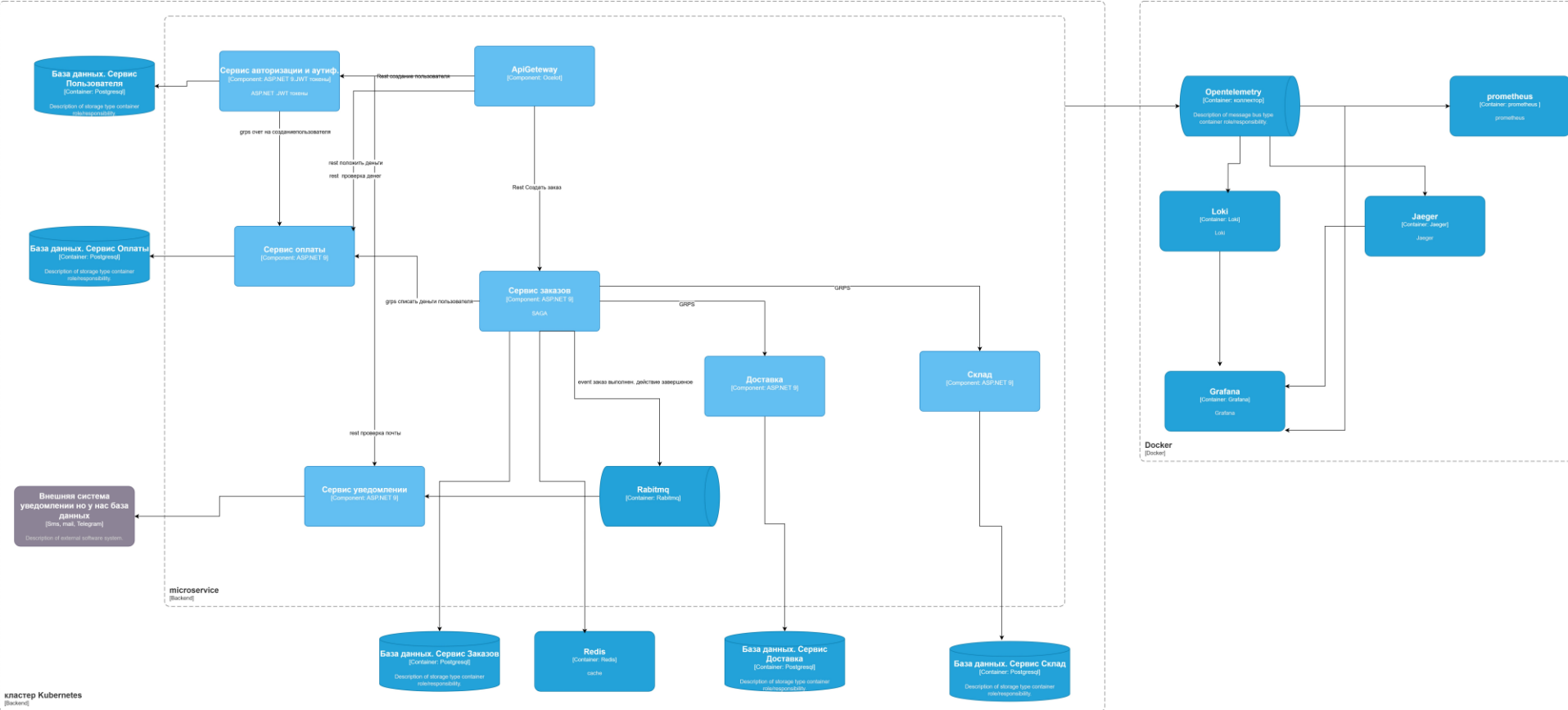
1. Декомпозиция с использованием Domain-Driven Design (DDD): микросервисы выстраиваются вокруг поддоменов (Subdomains), где каждый поддомен представляет собой ограниченный контекст (Bounded Context)
Чистая архитектура в микросервисах позволяет четко разделять слои — интерфейсов, бизнес-логики и инфраструктуры,
2. Для управления инфраструктурой микросервисов в Kubernetes используется Helm, позволяющий эффективно разворачивать сервисы, такую как RabbitMQ и PostgreSQL. Docker Compose применяется для компонентов мониторинга и наблюдаемости: OpenTelemetry, Prometheus, Jaeger, Loki и Grafana.
3. OpenTelemetry с экспортером данных в Prometheus для метрик, Jaeger для распределенной трассировки, Loki для логов, и визуализация через Grafana.
4. Микросервис одновременно прослушивает два порта для поддержки gRPC (HTTP/2) и REST (HTTP/1.1) интерфейсов, что обеспечивает гибкость взаимодействия с клиентами. Для асинхронного обмена сообщениями используется брокер RabbitMQ. Документирование REST API выполняется с помощью OpenAPI (Swagger), обеспечивая удобство работы и тестирования.



6. JWT Authentication - позволяет микросервисам безопасно проверять подлинность и права пользователей без центрального запроса в аутентифицированный сервис.
7. API Gateway Ocelot — легковесный и мощный API-шлюз на платформе .NET Core для маршрутизации, авторизации и управления трафиком между клиентами и микросервисами.
8. PostgreSQL — популярная open source реляционная база данных, обеспечивающая надёжное хранение данных и работу с транзакциями.
9. Redis — используется для реализации идемпотентности через Idempotency-Key, что предотвращает повторную обработку одинаковых запросов.
10. Распределённые транзакции с паттерном Saga — оркестратор управляет жизненным циклом заказов (order), обеспечивая согласованность процессов в распределённой системе.
11. CQRS с MediatR — разделение операций чтения и записи на складе для оптимизации производительности и упрощения архитектуры.
12. Event Sourcing с MediatR — хранение всех изменений состояний заказов для восстановления и аудита состояния системы.

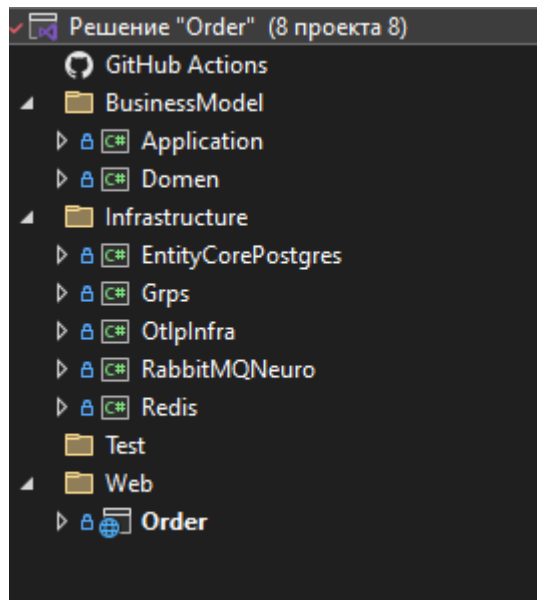
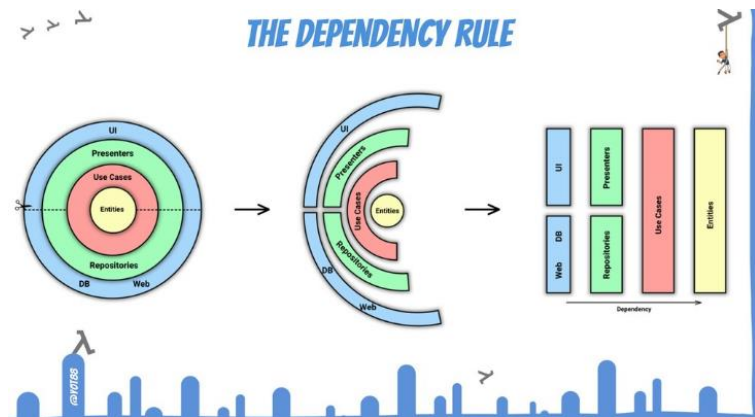


Что получилось



Clean Architecture

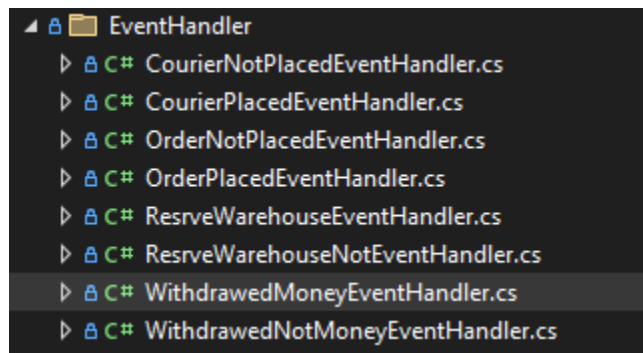
В каждом проекте соблюден



Сага

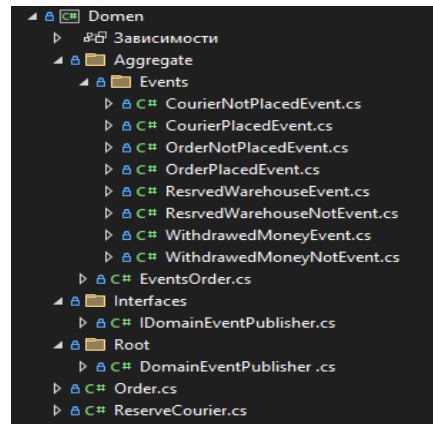
Контролирует транзакцию

Реализовал через доменное событие



event sourcing

- Событийная модель помогает bounded context правильно разделить
- Паттерн поддержания консистентности данных
- Помогает на код взглянуть с точки зрения завершённых событий
- Мониторинг(нейросеть) и разделение данных на чтение и запись(Cqrs)



Data Output

Messages

Notifications

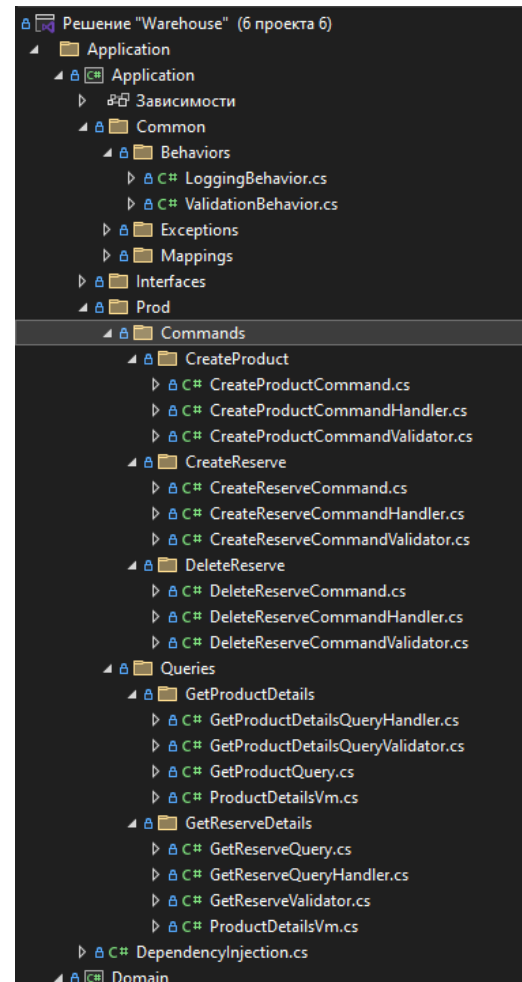
Showing rows: 1 to 8

Page No: 1

	Id [PK] integer	event_type text	aggregate_id uuid	aggregate_type text	data text	event_id integer
1	1	WithdrawedMoneyEvent	1a856435-40de-4c55-b2f9-0a6a38209...	Order	{'EventId':0,'AggregateId':'1a856435-40de-4c55-b2f9-0a6a382096d1','UserId':1,'Price':1111.0}	0
2	2	ResrvedWarehouseEvent	1a856435-40de-4c55-b2f9-0a6a38209...	Order	{'EventId':1,'AggregateId':'1a856435-40de-4c55-b2f9-0a6a382096d1','UserId':1,'Reserveld':2,'Price':1111.0}	1
3	3	OrderPlacedEvent	1a856435-40de-4c55-b2f9-0a6a38209...	Order	{'EventId':2,'AggregateId':'1a856435-40de-4c55-b2f9-0a6a382096d1','UserId':1,'Reserveld':2,'Price':1111.0,'OrderId':...	2
4	4	CourierPlacedEvent	1a856435-40de-4c55-b2f9-0a6a38209...	Order	{'EventId':3,'AggregateId':'1a856435-40de-4c55-b2f9-0a6a382096d1','Price':1111.0,'UserId':0}	3
5	5	WithdrawedMoneyEvent	b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d	Order	{'EventId':0,'AggregateId':'b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d','UserId':1,'Price':1112.0}	0
6	6	ResrvedWarehouseEvent	b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d	Order	{'EventId':1,'AggregateId':'b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d','UserId':1,'Reserveld':3,'Price':1112.0}	1
7	7	OrderPlacedEvent	b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d	Order	{'EventId':2,'AggregateId':'b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d','UserId':1,'Reserveld':3,'Price':1112.0,'OrderId':3}	2
8	8	CourierPlacedEvent	b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d	Order	{'EventId':3,'AggregateId':'b4c5f9aa-dfdc-4f70-94fc-b59c79c9024d','Price':1112.0,'UserId':0}	3

Cqrs (mediatR)

Огромный плюс разделение чтение и записи. Разгрузка системы, гибкость и стабильность в коде и т д
Обычно используется вместе event sourcing



JWT Authentication

Дает возможность децентрализовать авторизацию, может проверить токены без необходимости обращения к центральному серверу аутентификации

```
Ссылка 0
public static class JwtModule
{
    Ссылка 1
    public static IServiceCollection AddJwtAuthentication(this IServiceCollection self, string secretKey)
    {
        self.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
            .AddJwtBearer(t =>
            {
                t.TokenValidationParameters = new TokenValidationParameters()
                {
                    ValidateIssuer = false,
                    ValidateAudience = false,
                    ValidateLifetime = true,
                    ValidateIssuerSigningKey = true,
                    IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secretKey))
                };
            });

        return self;
    }
}
```

```
Ссылка 1
public string GenerateToken(Account account)
{
    string secret = Environment.GetEnvironmentVariable("SECRET*");

    var claimsList = new List<Claim>
    {
        new Claim("UserName", account.UserName),
        new Claim("Telephone", account.Telephone),
        new Claim("Id", account.Id.ToString())
    };

    var token = new JwtSecurityToken(
        expires: DateTime.UtcNow.AddHours(3),
        claims: claimsList,
        signingCredentials: new SigningCredentials(
            new SymmetricSecurityKey(Encoding.UTF8.GetBytes(secret)), SecurityAlgorithms.HmacSha256));

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Api Getway Ocelot Swagger

Единый центр. Удобная документация документации

Swagger
SMARTBEAR

Select a definition

- Order - v1
- UserService - v1
- Billing - v1
- Order - v1
- Notification - v1
- Warehouse - v1

Order **v1** OAS 3.0
</swagger/docs/v1/order>

Authorize

Order

POST /Order/PlaceOrder

Parameters

Try it out

Name	Description
amount	
number(float)	amount
(query)	
Idempotency-Key	Idempotency description
any	Idempotency-Key
(header)	

Responses

Code	Description	Links
200	OK	No links



Для отладки и быстрого запуска всех микросервисов

[illegible]

Helm

Хелм для быстрого запуска и оркестрации контейнеров на серверах



open telemetry

Используя коллектор помогает метрики,
логирование трассировки держать на другой
машине

```
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  batch:
    timeout: 1s

exporters:
  prometheus:
    endpoint: 0.0.0.0:8889

  otlp/jaeger:
    endpoint: jaeger:4317
    tls:
      insecure: true

  otlphttp/logs:
    endpoint: "http://loki:3100/otlp"
    tls:
      insecure: true

extensions:
  health_check:
  pprof:
    endpoint: :1888
  zpages:
    endpoint: :55679

service:
  extensions: [pprof, zpages, health_check]
  pipelines:
    metrics:
      receivers: [otlp]
      processors: [batch]
      exporters: [prometheus]
    traces:
      receivers: [otlp]
      exporters: [otlp/jaeger]
      processors: [batch]
    logs:
      receivers: [otlp]
      processors: [batch]
      exporters: [otlphttp/logs]
```

Ссылка 1

```
private static void AddLog(this IServiceCollection self, IHostBuilder hostBuilder, string otelConfig)
{
    self.AddSerilog();

    hostBuilder.UseSerilog((hostContext, services) =>
    {
        services
            .Enrich.FromLogContext()
            .ReadFrom.Configuration(hostContext.Configuration)
            .Enrich.WithProperty("EnvironmentName", hostContext.HostingEnvironment.EnvironmentName)
            .WriteTo.Console()
            .WriteTo.OpenTelemetry(config =>
            {
                config.IncludedData = IncludedData.TraceIdField | IncludedData.SpanIdField;
                config.Protocol = OtlpProtocol.Grpc;
                config.Endpoint = $"{otelConfig}";
                config.ResourceAttributes = new Dictionary<string, object>
                {{hostContext.HostingEnvironment.ApplicationName, hostContext.HostingEnvironment.ApplicationName }} };
            });
    });
}
```

Ссылка 1

```
private static void AddMetrics(this IServiceCollection self, string otelConfig)
{
    var otel = self.AddOpenTelemetry();

    otel.WithMetrics(metrics => metrics
        .AddAspNetCoreInstrumentation()
        .AddHttpClientInstrumentation()
        .AddMeter("Microsoft.AspNetCore.Hosting")
        .AddMeter("Microsoft.AspNetCore.Server.Kestrel")
        .AddMeter("System.Net.NameResolution")
        .AddRuntimeInstrumentation()
        .AddProcessInstrumentation()
        .AddOtlpExporter(option =>
        {
            option.Endpoint = new Uri(otelConfig);
            option.ExportProcessorType = ExportProcessorType.Batch;
            option.Protocol = OpenTelemetry.Exporter.OtlpExportProtocol.Grpc;
        });
});
}
```

```
private static void AddTracing(this IServiceCollection self, string otelConfig)
{
    var otel = self.AddOpenTelemetry();

    otel.WithTracing(tracing =>
    {
        tracing.ConfigureResource(res => res.AddService(serviceName: "Order", serviceVersion: "1.0.0"));
        tracing
            .AddAspNetCoreInstrumentation(option =>
            {
                option.RecordException = true;
                option.EnrichWithHttpRequest = (activity, req) =>
                {
                    activity.DisplayName = req.Method + " " + req.Path;
                    activity.SetTag("api", req.Method);
                });
            })
            .AddHttpClientInstrumentation()
            .AddSqlClientInstrumentation()
            .AddEntityFrameworkCoreInstrumentation()
            .AddGrpcClientInstrumentation()
            .AddOtlpExporter(option =>
            {
                option.Endpoint = new Uri(otelConfig);
                option.ExportProcessorType = ExportProcessorType.Batch;
                option.Protocol = OpenTelemetry.Exporter.OtlpExportProtocol.Grpc;
            })
            .AddConsoleExporter();
    });
}
```

Grps и rest прослушивание двух портов

Rest работает с api gateway. Grps с микросервисами

```
{
  "Kestrel": {
    "Endpoints": {
      "Grpc": {
        "Protocols": "Http2",
        "Url": "http://localhost:5104"
      },
      "webApi": {
        "Protocols": "Http1",
        "Url": "http://localhost:5105"
      }
    }
  }
}
```

```
var app = builder.Build();

app.UseMetricServer();
app.UseHttpMetrics();

app.UseSwagger();
app.UseSwaggerUI(c =>
{
  c.SwaggerEndpoint("/swagger/v1/swagger.json", "Billing");
  c.InjectStyleSheet("/swagger/custom.css");
  c.RoutePrefix = String.Empty;
});

app.UseHttpsRedirection();

app.MapGrpcService<AccountService>();
app.MapGrpcService<BillingService>();

app.MapControllers();
app.UseAuthentication();
app.UseAuthorization();
app.Run();
```

Ссылка 0

```
public static class GrpsSetting
```

```
{
```

Ссылка 1

```
public static IServiceCollection AddGrpsOrder(this IServiceCollection self, IConfiguration config)
```

```
{
```

```
    self.AddSingleton<GrpcLoggingInterceptor>();
```

```
    self.AddGrpcClient<BillingServiceGrpsClient>(o =>
```

```
    {
```

```
        o.Address = new Uri($"{config.GetSection("GrpsBillingConnection").Value}");
```

```
    }).AddInterceptor<GrpcLoggingInterceptor>();
```

```
    self.AddGrpcClient<WarehouseServiceGrpsClient>(o =>
```

```
    {
```

```
        o.Address = new Uri($"{config.GetSection("GrpsWarehouseConnection").Value}");
```

```
    }).AddInterceptor<GrpcLoggingInterceptor>();
```

```
    self.AddGrpcClient<DeliveryServiceGrpsClient>(o =>
```

```
    {
```

```
        o.Address = new Uri($"{config.GetSection("GrpsDeliveryConnection").Value}");
```

```
    }).AddInterceptor<GrpcLoggingInterceptor>();
```

```
    return self;
```

```
}
```

```
}
```

RabbitMq

Используем как событийную модель триггер завершённого события. Можем спокойно использовать асинхронно так как обратный ответ нам не нужен. Например отправка почты



Redis кэширование данных

```
Ссылка: 0
public static class RedisModule
{
    Ссылка: 1
    public static IServiceCollection AddRedisService(this IServiceCollection self, IConfiguration config)
    {
        var settings = config.GetSection("Redis").Value;

        self.AddStackExchangeRedisCache(options => {
            options.Configuration = settings;
            options.InstanceName = "Order";
        });
        self.AddScoped<ICache, Redis>();

        return self;
    }
}
```

Idempotency-Key

```
namespace Order.Idempotent
{
    [CSharp: 1]
    internal sealed class IdempotencyFilter(int cacheTimeInMinutes = 60)
        : IEndpointFilter
    {
        [CSharp: 0]
        public async ValueTask<object?> InvokeAsync(
            EndpointFilterInvocationContext context,
            EndpointFilterDelegate next)
        {
            if (context.HttpContext.GetEndpoint()?.Metadata.GetMetadata<IdempotencyAttribute>() == null)
            {
                return await next(context);
            }

            var headers = context.HttpContext.Request.Headers;

            // Parse the Idempotence-Key header from the request
            if (!headers.TryGetValue("Idempotency-Key", out var idempotencyheadersKey))
            {
                return Results.BadRequest("Invalid or missing Idempotence-Key header");
            }

            IDistributedCache cache = context.HttpContext
                .RequestServices.GetRequiredService<IDistributedCache>();

            // Check if we already processed this request and return a cached response (if it exists)
            var idempotencyKey = idempotencyheadersKey.ToString();
            string cacheKey = $"Idempotent_{idempotencyKey}";

            string? cachedResult = await cache.GetStringAsync(cacheKey);
            if (cachedResult is not null)
            {
                IdempotentResponse response = JsonSerializer.Deserialize<IdempotentResponse>(cachedResult)!;
                return new IdempotentResult(response.StatusCode, response.Value);
            }

            var result = await next(context);

            // Execute the request and cache the response for the specified duration
        }
    }
}
```

Jaeger

[Search](#) [Upload](#)

Service (5)

Billing

jaeger-all-in-one

Notification

Order

UserService

Lookback

Last Hour

Max Duration

e.g. 1.2s, 100ms, 500us

Min Duration

e.g. 1.2s, 100ms, 500us

Limit Results

20

Find Traces



20 Traces

Sort: [Most Recent](#) [Download Results](#) [Deep Dependency Graph](#)

Compare traces by selecting result items

☐	Order: GET Health 4f654f4	500µs
1 Span	Order (1)	Today 5:08:17 pm a few seconds ago
☐	Order: GET Health 7aff9db	698µs
1 Span	Order (1)	Today 5:08:17 pm a few seconds ago
☐	Order: opentelemetry.proto.collector.logs.v1.LogsService/Export 1830390	2.66ms
1 Span	Order (1)	Today 5:08:16 pm a few seconds ago
☐	Order: opentelemetry.proto.collector.logs.v1.LogsService/Export 1205216	2.11ms
1 Span	Order (1)	Today 5:08:15 pm a few seconds ago

Prometheus

 Prometheus

Alerts Status

Execute

Table Graph Explain

< Evaluation time >

Load time: 28ms Result series: 195

http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.005", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	0
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.01", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	0
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.025", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	0
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.05", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.075", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.1", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.25", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.5", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="0.75", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="1.0", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="2.5", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="5.0", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1
http_server_request_duration_seconds_bucket{exported_job="unknown_service:dotnet", group="otel-collector", http_request_method="GET", http_response_status_code="200", instance="otel-collector:8889", job="prometheus", le="7.5", network_protocol_version="1.1", otel_scope_name="Microsoft.AspNetCore.Hosting", url_scheme="http"}	1





```
> histogram_quantile(0.5, sum by(1e) (rate(http_server_request_duration_seconds_bucket(http_route=~"Order/PlaceOrder"){10m})))
```

Execute

Table Graph Explain

- 1h +

< End time >

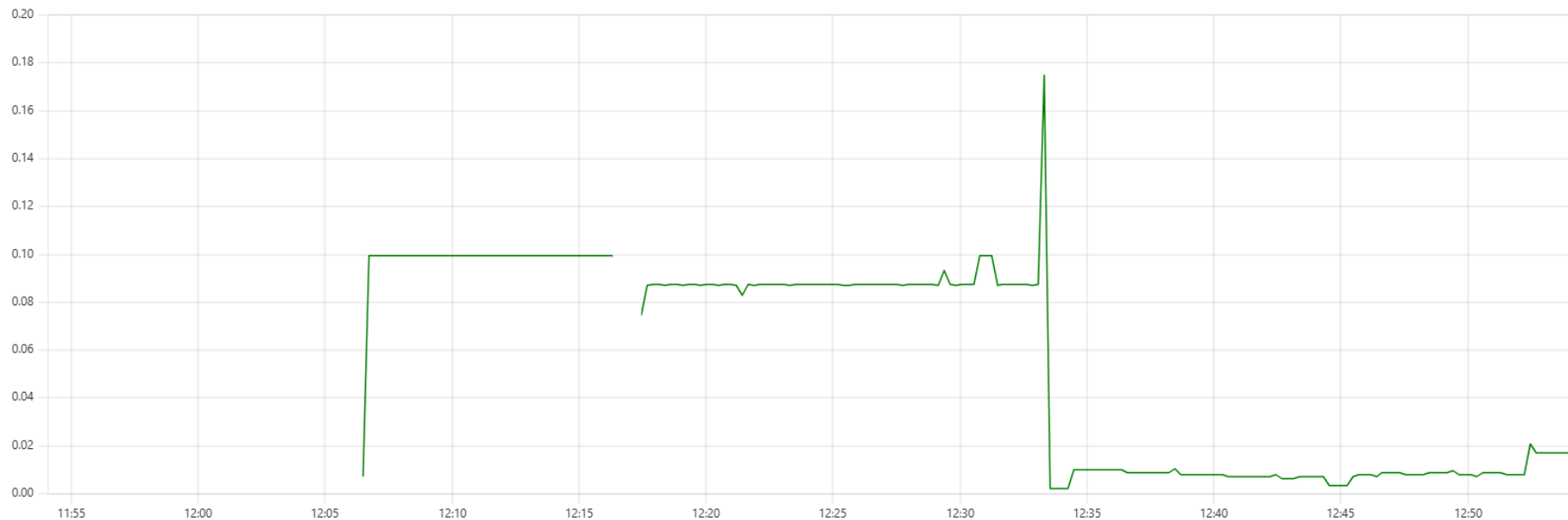
Medium res. ▾

Unstacked

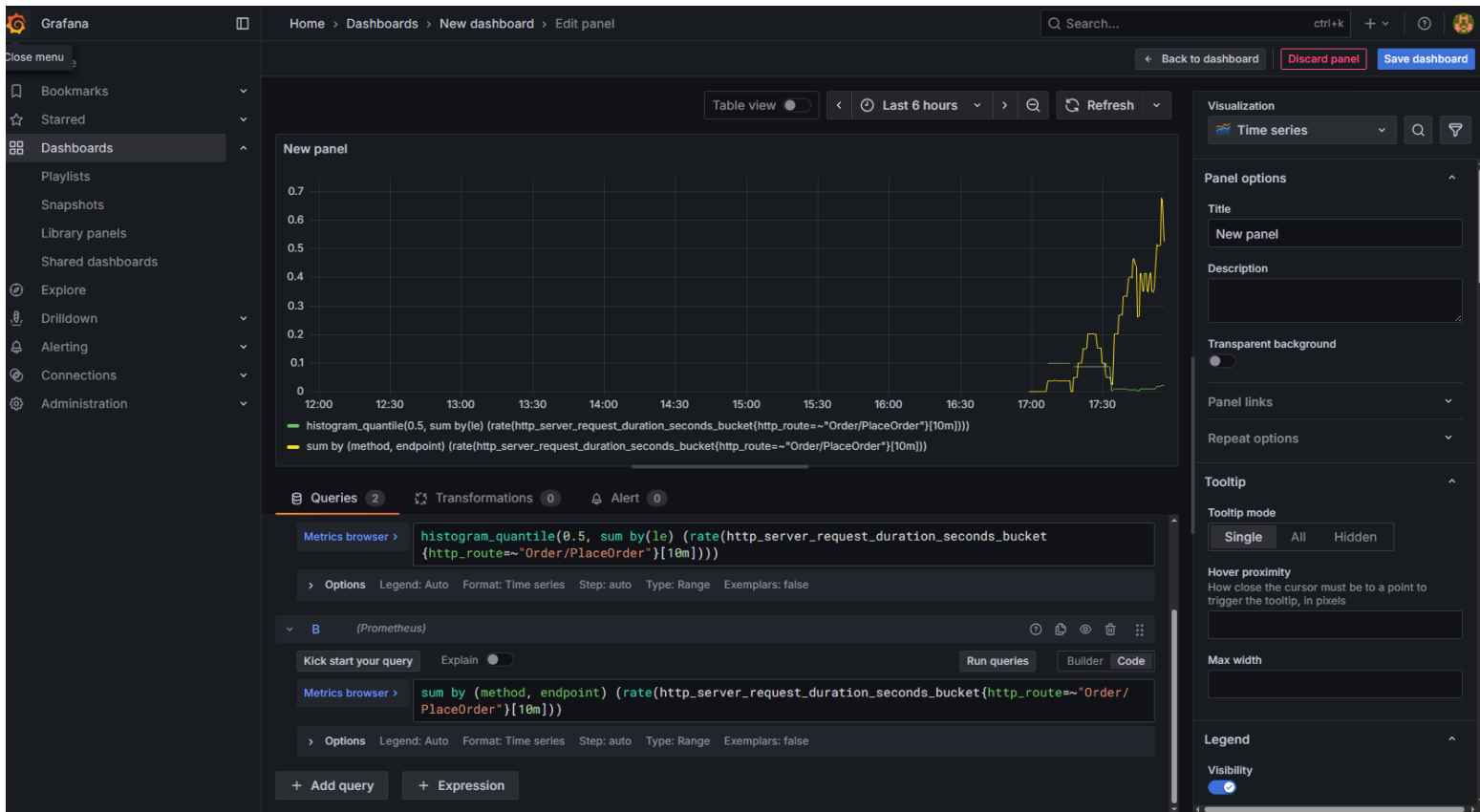
Stacked

Query notice

PromQL info: input to histogram_quantile needed to be fixed for monotonicity (see https://prometheus.io/docs/prometheus/latest/querying/functions/#histogram_quantile) for metric name "" (1:25)



Grafana



Логирование

- Interceptor grps, который будет логгировать request и response
- exception filter, который будет отлавливать все необработанные исключения и выдавать response
- middleware компоненты, которые будут логгировать request и response поступающих http запросов



Выводы

1. Мне не хватило кол во дз и показалось много в дз helm . Увлёк процесс очень сильно. Не хватило лайф кодинга. В Дз на практике нет CQRS, eventsource , opentelemetry, TDD, SI/SD gitlab, доменные события. Я попробовал дополнительно реализовать у себя в проекте. И получилось очень круто.
2. Микросервисов создал очень много. Можно экспериментировать.
3. Внедрил open telemetry на работе. Появились идеи что можно нового внедрить и улучшить



Вопросы и рекомендации



если есть вопросы



если вопросов нет



Спасибо за внимание!

OTUS

